

شكراً لكم



الكلية التطبيقية
الجامعة السعودية الإلكترونية
SAUDI ELECTRONIC UNIVERSITY
2011-1432



الكلية التطبيقية
الجامعة السعودية الإلكترونية
SAUDI ELECTRONIC UNIVERSITY
2011-1432

دبلوم الذكاء الاصطناعي التوليدي

اسم المقرر: البرمجة باستخدام بايثون

رمز المقرر: PYT103

الموضوع: الثاني عشر

الموضوع الثاني
عشر:
إعادة – NumPy
التشكيل، البتّ،
الأقنعة



الأهداف

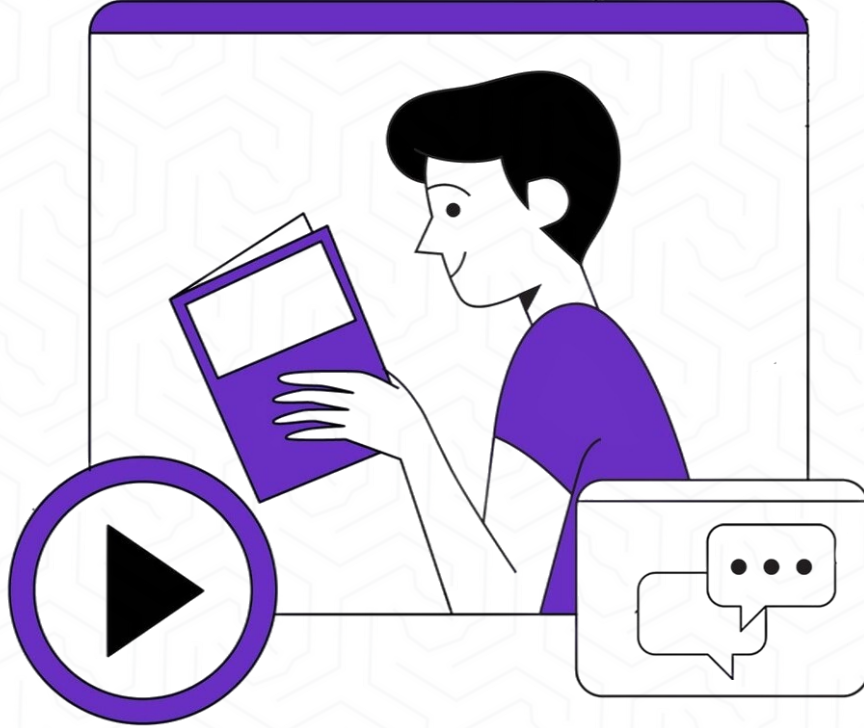
في نهاية هذا الدرس سيكون الطالب قادرًا على أن:

- ✓ يوضح مفهوم إعادة تشكيل المصفوفات Reshaping وتطبيقاته العملية.
- ✓ يستخدم دوال إعادة التشكيل المختلفة مثل `reshape()`, `flatten()`, `ravel()`.
- ✓ يفهم آلية البث Broadcasting في NumPy وقواعدها.
- ✓ يطبق العمليات الحسابية على مصفوفات بأشكال مختلفة.
- ✓ يستخدم الأقنعة المنطقية Boolean Masking لفلتر البيانات.
- ✓ يطبق الفهرسة المتقدمة Advanced Indexing والفهرسة الشرطية باستخدام دوال `where()`, `select()`, `choose()` للتحكم المتقدم.
- ✓ يقسم المصفوفات باستخدام `concatenate()`, `split()`, `stack()`.
- ✓ يطبق عمليات النقل والتبديل Transpose and Swapping وبناء تطبيقات عملية تجمع بين هذه التقنيات



وصف الدرس

في هذا الدرس سوف نتناول :



- التقنيات المتقدمة في مكتبة NumPy للتعامل مع البيانات متعددة الأبعاد بكفاءة عالية.
- مفهوم إعادة التشكيل Reshaping للمصفوفات وكيفية تغيير شكلها دون المساس بمحتواها.
- تطبيقات إعادة التشكيل في معالجة الصور، التعلم الآلي، وتحليل البيانات.
- مفهوم البث Broadcasting كآلية ذكية لتنفيذ العمليات الحسابية بين مصفوفات بأحجام مختلفة دون تكرار البيانات.
- الأقنعة المنطقية Boolean Masking كأداة قوية لفلتر البيانات واختيار العناصر بناءً على شروط منطقية محددة.
- أهمية هذه التقنيات في رفع كفاءة التحليل ومعالجة البيانات باستخدام NumPy



مقدمة إلى إعادة التشكيل Reshaping

المبدأ والفكرة:

إعادة التشكيل Reshaping هي عملية تغيير شكل المصفوفة (عدد الصفوف والأعمدة والأبعاد) دون تغيير البيانات نفسها. هذه العملية ضرورية في:

- تحضير البيانات للنماذج (مثل تحويل صورة إلى متجه)

- إعادة تنظيم البيانات للعمليات الحسابية
- التوافق مع متطلبات المكتبات الأخرى

الشروط المهمة:

- عدد العناصر يجب أن يبقى ثابتاً
- الترتيب الافتراضي: Row-major (C-style)

```
new_array = array.reshape(new_shape)
```


الموضوع الحادي عشر

مقدمة إلى إعادة التشكيل

Reshaping

```
import numpy as np
```

```
# إنشاء مصفوفة أحادية البعد
```

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
print("المصفوفة الأصلية:")
print(arr)
print(f"الشكل: {arr.shape}")
```

```
# (صفوف 4 × أعمدة 3) تحويل إلى مصفوفة 2
```

```
arr_2d = arr.reshape(3, 4)
print("\n4, 3 (بعد إعادة التشكيل إلى)")
print(arr_2d)
print(f"الشكل: {arr_2d.shape}")
```

```
# (بشكل مختلف) 4 صفوف × 3 أعمدة (D تحويل إلى مصفوفة 2
```

```
arr_2d_alt = arr.reshape(4, 3)
print("\n3, 4 (بعد إعادة التشكيل إلى)")
print(arr_2d_alt)
```

```
# (2 × 2 × 3) D تحويل إلى مصفوفة 3
```

```
arr_3d = arr.reshape(2, 2, 3)
print("\n3, 2, 2 (بعد إعادة التشكيل إلى)")
print(arr_3d)
print(f"الشكل: {arr_3d.shape}")
print(f"عدد الأبعاد: {arr_3d.ndim}")
```

```
# استخدام 1- للحساب التلقائي
```

```
# تعني: احسب هذا البعد تلقائياً بناءً على باقي الأبعاد 1-
```

```
arr_auto = arr.reshape(3, -1) # 3 صفوف، الأعمدة تُحسب تلقائياً
print("\n1-, 3 (بإستخدام 1-)")
print(arr_auto)
print(f"الشكل: {arr_auto.shape}") # 4, 3 (النتيجة)
```

```
arr_auto2 = arr.reshape(-1, 2) # عمودان، الصفوف تُحسب تلقائياً
```

```
print("\n2, 1- (بإستخدام 1-)")
print(arr_auto2)
print(f"الشكل: {arr_auto2.shape}") # 2, 6 (النتيجة)
```

```
# محاولة إعادة تشكيل خاطئة - سبب خطأ
```

```
try:
```

```
    wrong = arr.reshape(3, 5) # 3×5 = 15، لكن لدينا 12 عنصر فقط
except ValueError as e:
    print(f"خطأ: {e}")
```

```
# مثال عملي: تحويل بيانات طالب
```

```
# لدينا 4 طلاب، كل طالب له 3 درجات
```

```
grades = np.array([85, 90, 78, 92, 88, 95, 78, 85, 82, 88, 92, 90])
print("\n=== مثال عملي: درجات الطلاب ===")
print("البيانات الأصلية (مسطحة):", grades)
```

```
# إعادة تشكيل: 4 طلاب × 3 مواد
```

```
grades_matrix = grades.reshape(4, 3)
print("\n(مصفوفة الدرجات) 4 طلاب × 3 مواد")
print(grades_matrix)
print(f"درجات الطالب الأول: {grades_matrix[0]}")
print(f"درجات المادة الأولى لجميع الطلاب: {grades_matrix[:, 0]}")
```

Flatten و Ravel - تسطيح المصفوفات

المبدأ والفكرة:

التسطيح Flattening هو عملية تحويل مصفوفة متعددة الأبعاد إلى مصفوفة أحادية البعد (1D). هناك طريقتان رئيسيتان:

flatten():

- تُنشئ نسخة جديدة من البيانات
- التعديل عليها لا يؤثر على المصفوفة الأصلية
- تستهلك ذاكرة إضافية

ravel():

- تُنشئ view إن أمكن (أسرع)
- التعديل عليها قد يؤثر على المصفوفة الأصلية
- أكثر كفاءة في الذاكرة

الاستخدامات:

- تحضير البيانات لخوارزميات ML
- تحويل الصور إلى متجهات
- عمليات حسابية بسيطة



Flatten و Ravel

تسطيح المصفوفات

```
import numpy as np
```

```
# إنشاء مصفوفة ثنائية البعد
```

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
```

```
print("المصفوفة الأصلية:")
```

```
print(matrix)
```

```
print(f"الشكل: {matrix.shape}")
```

```
# ينشئ نسخة - flatten() استخدام 1
```

```
flattened = matrix.flatten()
```

```
print("\n=== باستخدام flatten() ===")
```

```
print("المصفوفة المسطحة:", flattened)
```

```
print(f"الشكل: {flattened.shape}")
```

```
# تعديل المصفوفة المسطحة
```

```
flattened[0] = 999
```

```
print("\nبعد تعديل flattened[0] = 999:")
```

```
print("المصفوفة المسطحة:", flattened)
```

```
print("المصفوفة الأصلية لم تتغير")
```

```
print(matrix) # لم تتأثر
```

```
# view ينشئ - ravel() استخدام 2
```

```
matrix2 = np.array([[10, 20, 30],
                   [40, 50, 60],
                   [70, 80, 90]])
```

```
raveled = matrix2.ravel()
```

```
print("\n=== باستخدام ravel() ===")
```

```
print("المصفوفة المسطحة:", raveled)
```

```
# تعديل المصفوفة المسطحة
```

```
raveled[0] = 999
```

```
print("\nبعد تعديل raveled[0] = 999:")
```

```
print("المصفوفة المسطحة:", raveled)
```

```
print("المصفوفة الأصلية تغيرت!")
```

```
print(matrix2) # تأثرت
```

نقل المصفوفات Transpose

المبدأ والفكرة:

النقل Transpose هو عملية قلب المصفوفة بحيث تصبح الصفوف أعمدة والأعمدة صفوف. في المصفوفات الرياضية، النقل يُرمز له بـ A^T .

الطرق المختلفة:

- T - اختصار سريع
- $transpose()$ - دالة كاملة

الاستخدامات:

- عمليات الجبر الخطي
- تحضير البيانات
- عمليات ضرب المصفوفي

```
import numpy as np
```

```
# مصفوفة بسيطة 2
```

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
```

```
print("المصفوفة الأصلية 3x2")
print(matrix)
print(f"الشكل: {matrix.shape}")
```

```
# استخدام .T
```

```
transposed = matrix.T
print("\n=== باستخدام .T ===")
print("المصفوفة المنقولة 2x3")
print(transposed)
print(f"الشكل: {transposed.shape}")
```

```
# استخدام transpose()
```

```
transposed2 = matrix.transpose()
print("\n=== باستخدام transpose() ===")
print(transposed2)
```


مقدمة إلى البث Broadcasting

المبدأ والفكرة:

البث Broadcasting هو آلية قوية في NumPy تسمح بإجراء العمليات الحسابية بين مصفوفات ذات أشكال مختلفة دون الحاجة لنسخ البيانات يدوياً.

الفوائد:

- كود أنظف وأقصر
- أداء أفضل (لا حاجة للحلقات)
- استخدام أقل للذاكرة

قواعد البث:

1. إذا كانت المصفوفتان لهما نفس عدد الأبعاد، يتم المقارنة بُعد بُعد.
2. الأبعاد متوافقة إذا كانت:
 1. متساوية، أو
 2. أحدها يساوي 1

مقدمة إلى البث
Broadcasting

```
import numpy as np
```

```
# مثال بسيط: إضافة قيمة ثابتة لمصفوفة
print("=== مثال 1: إضافة قيمة ثابتة ===")
arr = np.array([1, 2, 3, 4, 5])
print("المصفوفة:", arr)
```

```
# إضافة 10 لكل عنصر (البث التلقائي)
result = arr + 10
print("بعد إضافة 10:", result)
# NumPy [10, 10, 10, 10, 10] لتصبح [10, 10, 10, 10, 10]
```

```
# مثال 2: عمليات بين مصفوفة وقيمة
print("=== مثال 2: عمليات متعددة ===")
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])
```

```
print("المصفوفة الأصلية:")
print(arr)
```

```
print("ضرب 2 ×:")
print(arr * 2)
```

```
print("قسمة 3 ÷:")
print(arr / 3)
```

```
print("أس 2:")
print(arr ** 2)
```

```
# مثال 3: إضافة صف إلى مصفوفة
print("=== مثال 3: إضافة صف ===")
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
```

```
row = np.array([10, 20, 30])
```

```
print("المصفوفة 3×3:")
print(matrix)
print(f"الشكل: {matrix.shape}")
```

```
print("الصف 3:")
print(row)
print(f"الشكل: {row.shape}")
```

```
# الجمع - الصف يُبث لكل صف في المصفوفة
result = matrix + row
print("النتيجة:")
print(result)
print("الصف [10, 20, 30] أُضيف لكل صف في المصفوفة")
```

```
# مثال 4: إضافة عمود إلى مصفوفة
print("=== مثال 4: إضافة عمود ===")
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
```

```
column = np.array([[10],
                   [20],
                   [30]])
```

```
print("المصفوفة 3×3:")
print(matrix)
```

```
print("العمود 1×3:")
print(column)
print(f"الشكل: {column.shape}")
```

مقدمة إلى الأقنعة المنطقية Boolean Masking

المبدأ والفكرة:

الأقنعة المنطقية Boolean Masking هي تقنية قوية لفلتر واختيار عناصر محددة من المصفوفة بناءً على شروط منطقية.

كيف تعمل:

1. نطبق شرط منطقي على المصفوفة
2. النتيجة: مصفوفة منطقية True/False
3. نستخدم هذه المصفوفة "كقناع"

لاختيار العناصر

الاستخدامات:

- فلتر البيانات
- البحث الشرطي
- تنظيف البيانات
- التحليل الإحصائي الشرطي

مقدمة إلى الأقنعة المنطقية Boolean Masking

```
import numpy as np
```

مصفوفة بسيطة

```
arr = np.array([10, 25, 30, 15, 40, 5, 35, 20])
```

```
print("=== المصفوفة الأصلية ===")
print(arr)
```

إنشاء قناع منطقي 1.

```
print("\n=== إنشاء قناع منطقي ===")
```

شرط: العناصر الأكبر من 20

```
mask = arr > 20
print(f"القناع (arr > 20):")
print(mask)
print(f"نوع البيانات: {mask.dtype}")
```

استخدام القناع لاختيار العناصر 2.

```
print("\n=== استخدام القناع ===")
filtered = arr[mask]
print(f"20 > العناصر: {filtered}")
```

أو مباشرة بدون متغير وسيط

```
print(f"20 < العناصر: {arr[arr < 20]}")
```

شروط منطقية متعددة 3.

```
print("\n=== شروط منطقية متعددة ===")
```

AND: &

العناصر بين 15 و 35

```
condition = (arr >= 15) & (arr <= 35)
print(f"35 و 15 بين العناصر: {arr[condition]}")
```

OR: |

العناصر أقل من 15 أو أكبر من 30

```
condition = (arr < 15) | (arr > 30)
print(f"30 > أو 15 < العناصر: {arr[condition]}")
```

NOT: ~

العناصر التي ليست بين 15 و 30

```
condition = ~(arr >= 15) & (arr <= 30)
print(f"30, 15 خارج نطاق: {arr[condition]}")
```

عدّ العناصر المطابقة 4.

```
print("\n=== عدّ العناصر ===")
count_greater_20 = (arr > 20).sum() # True = 1, False = 0
print(f"20 > عدد العناصر: {count_greater_20}")
```

```
count_between = ((arr >= 15) & (arr <= 35)).sum()
```

```
print(f"35 و 15 بين العناصر: {count_between}")
```

التحقق من الشروط 5.

```
print("\n=== التحقق من الشروط ===")
```

هل جميع العناصر > 0؟

```
all_positive = np.all(arr > 0)
print(f"{all_positive} هل جميع العناصر موجبة؟")
```

هل يوجد عنصر واحد على الأقل > 30؟

```
any_greater_30 = np.any(arr > 30)
print(f"{any_greater_30} هل يوجد عنصر > 30؟")
```

مصفوفات ثنائية البعد 6.

```
print("\n=== مصفوفات ثنائية البعد ===")
```

```
matrix = np.array([[10, 25, 30],
                   [15, 40, 5],
                   [35, 20, 12]])
```

```
print("المصفوفة:")
print(matrix)
```

قناع العناصر > 20

```
mask = matrix > 20
print("\n20 (> القناع:")
print(mask)
```

(D ستكون مصفوفة 1) اختيار العناصر

```
filtered = matrix[mask]
print(f"\n20 > العناصر: {filtered}")
```

مثال عملي: درجات الطلاب

```
print("\n=== مثال عملي: درجات الطلاب ===")
```

درجات 10 طلاب

```
grades = np.array([85, 92, 78, 65, 88, 45, 95, 72, 55, 90])
```

```
print(f"الدرجات: {grades}")
```

الطلاب الناجحون (≥ 60)

```
passing = grades[grades >= 60]
```

```
print(f"\n60(>= الناجحون: {passing})")
```

```
print(f"عدد الناجحين: {len(passing)}")
```

الطلاب الراسبون (< 60)

```
failing = grades[grades < 60]
```

```
print(f"\n60(< الراسبون: {failing})")
```

```
print(f"عدد الراسبين: {len(failing)}")
```

المتفوقون (≥ 90)

```
excellent = grades[grades >= 90]
```

```
print(f"\n90(>= المتفوقون: {excellent})")
```

الدرجات بين 70 و 85

```
middle_range = grades[(grades >= 70) & (grades < 85)]
```

```
print(f"\n85 و 70 الدرجات بين: {middle_range}")
```



تعديل العناصر باستخدام الأقنعة

المبدأ والفكرة:

الأقنعة لا تُستخدم فقط لاختيار العناصر، بل أيضاً لتعديلها بناءً على شروط.

```
import numpy as np
```

```
# مصفوفة درجات
```

```
grades = np.array([85, 45, 92, 35, 78, 65, 95, 58, 72, 50])
```

```
print("=== الدرجات الأصلية ===")
```

```
print(grades)
```

```
# تعديل العناصر المطابقة للشرط 1.
```

```
print("\n=== مثال 1: رفع الدرجات المنخفضة ===")
```

```
# رفع الدرجات الأقل من 60 إلى 60 (الدرجة الدنيا للنجاح)
```

```
grades_copy = grades.copy()
```

```
grades_copy[grades_copy < 60] = 60
```

```
print("بعد رفع الدرجات < 60 إلى 60")
```

```
print(grades_copy)
```

```
# إضافة قيمة للعناصر المطابقة 2.
```

```
print("\n=== مثال 2: درجات إضافية ===")
```

```
grades_copy = grades.copy()
```

```
# إضافة 5 درجات للطلاب الذين حصلوا على 90
```

```
grades_copy[grades_copy >= 90] += 5
```

```
# لكن لا نتجاوز 100
```

```
grades_copy[grades_copy > 100] = 100
```

```
print("بعد إضافة 5 درجات للمتفوقين")
```

```
print(grades_copy)
```

```
# 3. بديل أنيق - np.where() استخدام
```

```
print("\n=== np.where() مثال 3: استخدام ===")
```

```
# np.where(condition, value_if_true, value_if_false)
```

```
# إذا الدرجة >= 60 اكتبها كما هي، وإلا اكتبها 60
```

```
adjusted = np.where(grades >= 60, grades, 60)
```

```
print("np.where() باستخدام")
```

```
print(adjusted)
```


تعديل العناصر باستخدام الأقنعة (يتبع)

```
# مثال آخر: تصنيف النجاح/الرسوب
status = np.where(grades >= 60, "راسب", "راسب")
print("\nالحالة:")
print(status)

# 4. تعديلات متعددة الشروط
print("\n=== مثال 4: تعديلات متعددة ===")

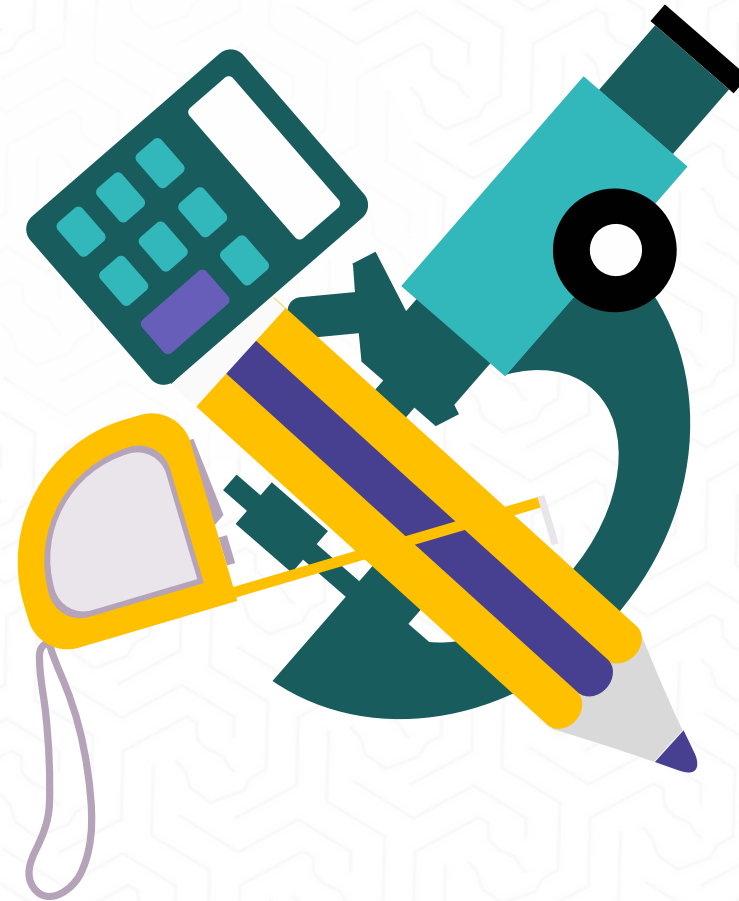
grades_copy = grades.copy()

# منح درجات إضافية حسب الفئة
# 90+: +5
# 80-89: +3
# 70-79: +2
# أقل من 70: +70

grades_copy[grades_copy >= 90] += 5
grades_copy[(grades_copy >= 80) & (grades_copy < 90)] += 3
grades_copy[(grades_copy >= 70) & (grades_copy < 80)] += 2

# التأكد من عدم تجاوز 100
grades_copy[grades_copy > 100] = 100

print("\n:بعد الدرجات الإضافية المتدرجة")
print(grades_copy)
```





Python for Everybody: Exploring Data in Python 3 Book by Charles Severance.

لکم شکراً





الكلية التطبيقية

الجامعة السعودية الإلكترونية

SAUDI ELECTRONIC UNIVERSITY

2011-1432

دبلوم الذكاء الاصطناعي التوليدي

اسم المقرر: البرمجة باستخدام بايثون

رمز المقرر: PYT103

الموضوع : العاشر



الموضوع العاشر: البرمجة الكينونية الأساسية

الجامعة الوطنية

الأهداف

في نهاية هذا الدرس سيكون الطالب قادرًا على أن:

✓ يفهم المبادئ الأساسية للبرمجة الكينونية Object-Oriented Programming

✓ ينشئ الأصناف Classes والكائنات Objects في Python

✓ يستخدم الخصائص Attributes والدوال Methods

✓ يوضح مفهوم البناء (Constructor) والمُنشئ __init__

✓ يطبق مبدأ التغليف Encapsulation الأساسي

✓ يميز بين متغيرات الصنف ومتغيرات الكائن

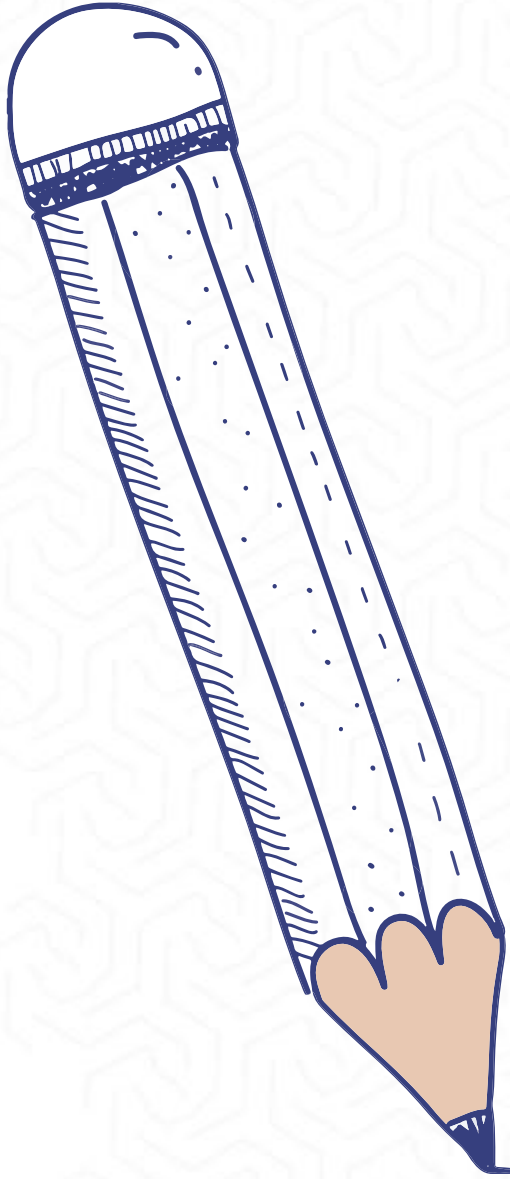
✓ ينشئ برامج بسيطة باستخدام البرمجة الكينونية



وصف الدرس

في هذا الدرس سنتناول:

- مفهوم البرمجة الكينونية.
- تصميم الأصناف في لغة Python وإنشاء الكائنات منها.
- إضافة الخصائص والدوال داخل الأصناف لتحديد سلوك الكائنات.
- استخدام الدالة الخاصة `__init__` في تهيئة الكائنات عند إنشائها.
- تطبيق عملي للمفاهيم النظرية من خلال أمثلة واقعية تساعد على الفهم والتطبيق.
- تعزيز المهارات البرمجية من خلال بناء برامج منظمة باستخدام مبادئ البرمجة الكينونية.



الموضوع العاشر | ما هي البرمجة الكينونية؟

المفهوم:

- البرمجة الكينونية OOP هي طريقة لتنظيم الكود.
- تعتمد على "الكائنات" التي تجمع البيانات والوظائف معاً.
- تحاكي العالم الحقيقي (مثال: سيارة لها خصائص ووظائف).
- الفوائد:
- تنظيم أفضل للكود.
- سهولة إعادة الاستخدام.
- صيانة أسهل للبرامج الكبيرة.



الموضوع العاشر | المفاهيم الأساسية

المصطلحات الرئيسية:

- **الصف:** Class قالب أو مخطط لإنشاء الكائنات.
- **الكائن:** Object نسخة محددة من الصف
- **الخصائص:** Attributes البيانات المخزنة في الكائن
- **الدوال:** Methods الوظائف المرتبطة بالكائن

مثال من الحياة:

- **الصف:** مخطط السيارة
- **الكائن:** سيارتك الشخصية
- **الخصائص:** اللون، الموديل، السرعة
- **الدوال:** تشغيل، إيقاف، تسريع

الفكرة: نبدأ بأبسط صنف ممكن - صنف فارغ

التوضيح:

- Class كلمة محجوزة لتعريف صنف جديد
- Car اسم الصنف (يبدأ بحرف كبير بالاتفاق)
- Pass تعني "لا شيء الآن" (صنف فارغ)
- my_car كائن (نسخة) من الصنف Car

• #إنشاء صنف فارغ

class Car: •

pass •

• #إنشاء كائن من الصنف

my_car = Car() •

print(type(my_car)) # <class '__main__.Car'> •

الفكرة: الخصائص هي البيانات التي يحملها الكائن

التوضيح:

- Wheels خاصية مشتركة بين جميع الكائنات.
- يمكن الوصول إليها عبر اسم الكائن متبوعاً بنقطة.

class Car: •

خاصية على مستوى الصنف •

wheels = 4 •

#إنشاء كائن •

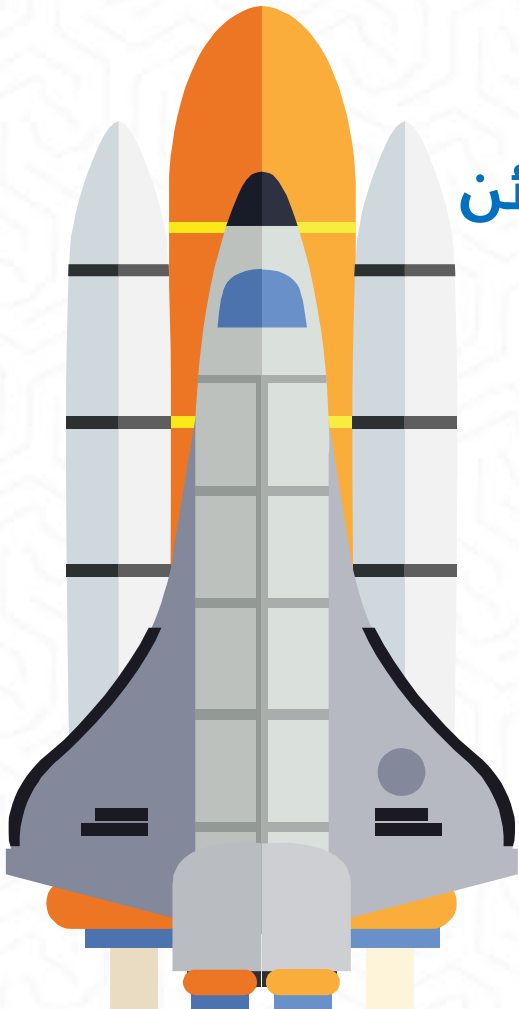
my_car = Car() •

print(my_car.wheels) # 4 •

#تغيير الخاصية لهذا الكائن فقط •

my_car.wheels = 3 •

print(my_car.wheels) # 3 •



الفكرة: دالة خاصة تُنفذ تلقائياً عند إنشاء كائن جديد

- `__init__` تُنفذ تلقائياً عند إنشاء الكائن
- `Self` تشير إلى الكائن الحالي
- نمرر القيم عند إنشاء الكائن
- ```
class Car:
```
- ```
# دالة البناء
```
- ```
def __init__(self, color, model):
```
- ```
# خاصية اللون self.color = color
```
- ```
خاصية الموديل self.model = model
```
- ```
# إنشاء كائن مع تمرير القيم
```
- ```
my_car = Car("أحمر", "2024")
```
- ```
# أحمر print(my_car.color)
```
- ```
2024 print(my_car.model)
```

### الفكرة: self هي إشارة للكائن نفسه

#### التوضيح:

- كل كائن له نسخته الخاصة من name
- self تميز بين student1.name و student2.name

```
class Student:
 def __init__(self, name):
 # اسم هذا الطالب المحدد
 self.name = name
```

```
إنشاء طالبين
student1 = Student("أحمد")
student2 = Student("فاطمة")
```

```
print(student1.name) # أحمد
print(student2.name) # فاطمة
```

### الفكرة: الدوال هي الإجراءات التي يمكن للكائن القيام بها

ملاحظة:

- الدوال داخل الصنف تأخذ self كأول معامل
- يمكنها تعديل خصائص الكائن

```
class Car:
 def __init__(self, color):
 self.color = color
 self.speed = 0 # السرعة الابتدائية

 # دالة لزيادة السرعة
 def accelerate(self):
 self.speed += 10
 print(f"السرعة الآن: {self.speed}")
```

```
my_car = Car("أزرق")
my_car.accelerate() # السرعة الآن: 10
my_car.accelerate() # السرعة الآن: 20
```



## الفكرة: الدوال يمكنها استقبال معاملات إضافية غير self



```
class BankAccount:
 def __init__(self, balance):
 self.balance = balance
```

# دالة الإيداع تستقبل المبلغ

```
def deposit(self, amount):
 self.balance += amount
 print(f"تم الإيداع: {amount}")
 print(f"الرصيد الجديد: {self.balance}")
```

```
account = BankAccount(1000)
```

```
account.deposit(500)
```

# 500 تم الإيداع

# 1500 الرصيد الجديد

## الموضوع العاشر مثال تطبيقي

### صنف الطالب:

```
class Student:
def __init__(self, name, grade):
 self.name = name
 self.grade = grade
```

```
دالة لعرض المعلومات
def display_info(self):
print(f"الاسم: {self.name}")
print(f"الدرجة: {self.grade}")
```

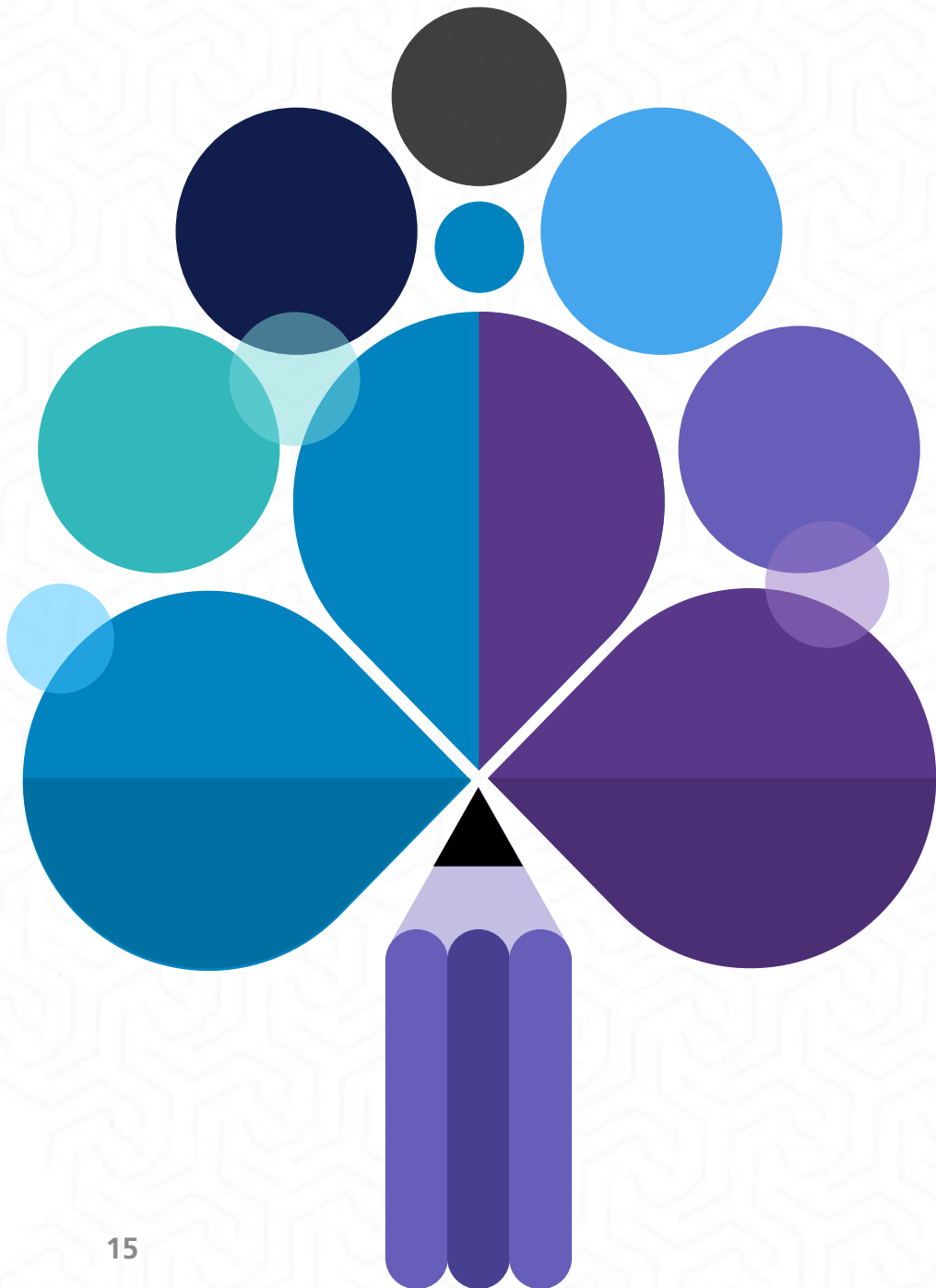
```
دالة للتحقق من النجاح
def is_passing(self):
return self.grade >= 50
```

```
استخدام الصنف
student = Student("علي", 75)
student.display_info()
علي: الاسم
75: الدرجة
```

```
if student.is_passing():
 print("ناجح") # ناجح
```

## الموضوع العاشر | متغيرات الصنف vs متغيرات الكائن

### الفكرة: هناك نوعان من المتغيرات



```
class School:
 # مشترك بين الجميع - متغير صنف
 school_name = "مدرسة النور"

 def __init__(self, student_name):
 # خاص بكل طالب - متغير كائن
 self.student_name = student_name

student1 = School("أحمد")
student2 = School("سارة")
```

```
متغير الصنف متشابه
print(student1.school_name) # مدرسة النور
print(student2.school_name) # مدرسة النور
```

```
متغير الكائن مختلف
print(student1.student_name) # أحمد
print(student2.student_name) # سارة
```

## الفائدة: مفيد لعد الكائنات أو مشاركة البيانات



```
class Counter:
 # عداد مشترك بين جميع الكائنات
 total_count = 0
```

```
 def __init__(self):
 # زيادة العداد عند إنشاء كائن جديد
 Counter.total_count += 1
```

```
 # إنشاء عدة كائنات
 c1 = Counter()
 c2 = Counter()
 c3 = Counter()
```

```
 # العداد يزيد مع كل كائن جديد
 print(Counter.total_count) # 3
```



## تطبيق - صنف المستطيل

```
class Rectangle:
def __init__(self, width, height):
 self.width = width
 self.height = height
```

```
حساب المساحة
def area(self):
 return self.width * self.height
```

```
حساب المحيط
def perimeter(self):
 return 2 * (self.width + self.height)
```

```
إنشاء مستطيل
rect = Rectangle(5, 3)
print(f"المساحة: {rect.area()}") # المساحة: 15
print(f"المحيط: {rect.perimeter()}") # المحيط: 16
```

## الموضوع العاشر | التغليف الأساسي - المتغيرات الخاصة

### الفكرة: نستخدم \_\_ قبل اسم المتغير لجعله خاصاً

```
class Account:
 def __init__(self, balance):
 # __ يبدأ بـ (متغير خاص)
 self.__balance = balance

 # دالة للوصول للرصيد
 def get_balance(self):
 return self.__balance

 # دالة لتعديل الرصيد بشروط
 def withdraw(self, amount):
 if amount <= self.__balance:
 self.__balance -= amount
 return True
 return False
```

```
account = Account(1000)
print(account.__balance) # لا يمكن الوصول مباشرة - خطأ
print(account.get_balance()) # 1000 الطريقة الصحيحة
```

### المفهوم الأساسي

#### المتغيرات الخاصة

هي متغيرات نريد إخفاءها عن الوصول المباشر من خارج الصنف، وذلك لحماية البيانات والتحكم في طريقة تعديلها.

# الموضوع العاشر | أنواع المتغيرات من حيث الوصول

## الأنواع

class Example:

def \_\_init\_\_(self):

self.public\_var = "متغير عام" # يمكن الوصول إليه من أي مكان -عام

self.\_protected\_var = "متغير محمي" # اتفاق أنه للاستخدام الداخلي -محمي

self.\_\_private\_var = "متغير خاص" # مخفي بشكل حقيقي -خاص

obj = Example()

print(obj.public\_var) # يعمل ✓

print(obj.\_protected\_var) # يعمل لكن غير مستحسن ✓

# print(obj.\_\_private\_var) # لا يمكن الوصول -خطأ ✗

:  
١- المتغير العام Public بدون أي شرطة سفلية الاستخدام:

للبيانات التي يجب أن تكون متاحة للجميع عندما لا توجد قيود على التعديل

٢- المتغير المحمي Protected شرطة سفلية واحدة - الاستخدام:

- مجرد اتفاق بين المبرمجين
- Python لا تمنع الوصول فعلياً
- يعني "استخدمني بحذر - للاستخدام الداخلي"

٣- التفصيل: المتغير الخاص Private شرطتان سفليتان - الاستخدام:

تقوم بايثون بتنشويه للاسم Name mangling  
\_\_private\_var

يصبح داخلياً هذا يمنع الوصول العرضي للمتغير.

## ١. حماية البيانات

```
class Temperature:
 def __init__(self):
 self.__celsius = 0

 def set_celsius(self, value):
 # التحقق من صحة القيمة
 if value < -273.15: # الصفر المطلق
 print("!درجة حرارة غير ممكنة")
 return
 self.__celsius = value

 def get_celsius(self):
 return self.__celsius

temp = Temperature()
temp.set_celsius(-300) # !درجة حرارة غير ممكنة
لو كانت عامة، لكان يمكن وضع قيمة خاطئة مباشرة
```



## الموضوع العاشر | لماذا نستخدم المتغيرات الخاصة؟

### ٢. التحكم في التعديل

### ٣. سهولة الصيانة

يمكن تغيير طريقة التخزين الداخلية دون التأثير على الكود الخارجي

#### جدول المقارنة السريع

| الاستخدام          | الوصول من خارج الصنف        | الرمز      | النوع |
|--------------------|-----------------------------|------------|-------|
| بيانات عامة للجميع | ✓ مسموح                     | self.var   | عام   |
| للاستخدام الداخلي  | ✓ مسموح (اتفاقياً)<br>ممنوع | self._var  | محمي  |
| بيانات حساسة جداً  | X ممنوع (مخفي)              | self.__var | خاص   |

```
class Age:
 def __init__(self, age):
 self.__age = age

 def set_age(self, new_age):
 if 0 <= new_age <= 150:
 self.__age = new_age
 else:
 print("عمر غير منطقي")

 def get_age(self):
 return self.__age

person = Age(25)
person.set_age(200) # عمر غير منطقي
print(person.get_age()) # لم يتغير - 25
```

## الفكرة: دالة خاصة لتحديد كيفية طباعة الكائن



```
class Person:
 def __init__(self, name, age):
 self.name = name
 self.age = age

 # تحديد طريقة الطباعة
 def __str__(self):
 return f"{self.name} - العمر: {self.age}"
```

```
person = Person("محمد", 25)
```

```
ستطبع الذاكرة __str__ بدون
تطبع النص المخصص __str__ مع
print(person) # العمر: 25 - محمد
```

### عند كتابة الأصناف:

#### ١. التسمية:

١. أسماء الأصناف تبدأ بحرف كبير
٢. استخدم أسماء واضحة ومعبرة

#### ٢. التنظيم:

١. `__init__` في البداية
٢. ثم الدوال العامة
٣. الدوال الخاصة في النهاية

#### ٣. البساطة:

١. كل صنف له مسؤولية واحدة واضحة
٢. لا تضع كل شيء في صنف واحد

مثال على صنف منظم جيداً #

```
class Car:
```

```
 def __init__(self, brand, model):
```

```
 self.brand = brand
```

```
 self.model = model
```

```
 self.speed = 0
```

```
 def accelerate(self):
```

```
 self.speed += 10
```

```
 def brake(self):
```

```
 self.speed = max(0, self.speed - 10)
```

```
 def __str__(self):
```

```
 return f"{self.brand} {self.model}"
```

## الموضوع العاشر | برنامج لتطبيق شامل- نظام الطلاب في المدرسة



```
class Student:
 # عدد للطلاب
 student_count = 0
```

```
def __init__(self, name, student_id):
 self.name = name
 self.student_id = student_id
self.grades = {} # قاموس المواد والدرجات
Student.student_count += 1
```

**# إضافة درجة لمادة**

```
def add_grade(self, subject, grade):
 self.grades[subject] = grade
```

**# حساب المعدل العام**

```
def calculate_average(self):
 if not self.grades:
 return 0
return sum(self.grades.values()) / len(self.grades)
```

**# عرض التقرير**

```
def report(self):
 print(f"اسم الطالب: {self.name}")
 print(f"الرقم: {self.student_id}")
 print("الدرجات:")
 for subject, grade in self.grades.items():
 print(f" {subject}: {grade}")
 print(f"المعدل: {self.calculate_average():.2f}")
```

```
def __str__(self):
 return f"الطالب: {self.name} ({self.student_id})"
```



## الموضوع العاشر | برنامج لتطبيق شامل- نظام الطلاب في المدرسة



### # اختبار النظام

```
student1 = Student("أحمد علي", "S001")
student1.add_grade("رياضيات", 85)
student1.add_grade("علوم", 90)
student1.add_grade("لغة عربية", 88)
```

```
student2 = Student("فاطمة محمد", "S002")
student2.add_grade("رياضيات", 92)
student2.add_grade("علوم", 87)
```

### # عرض التقارير

```
student1.report()
print("\n" + "="*30 + "\n")
student2.report()
print(f"\nإجمالي عدد الطلاب: {Student.student_count}")
```

## الموضوع العاشر | برنامج لتطبيق شامل- نظام الطلاب في المدرسة

### النتائج:

أحمد علي :اسم الطالب

S001 :الرقم

الدرجات:

85 : رياضيات

90 : علوم

88 : لغة عربية

87.67 : المعدل

فاطمة محمد :اسم الطالب

S002 :الرقم

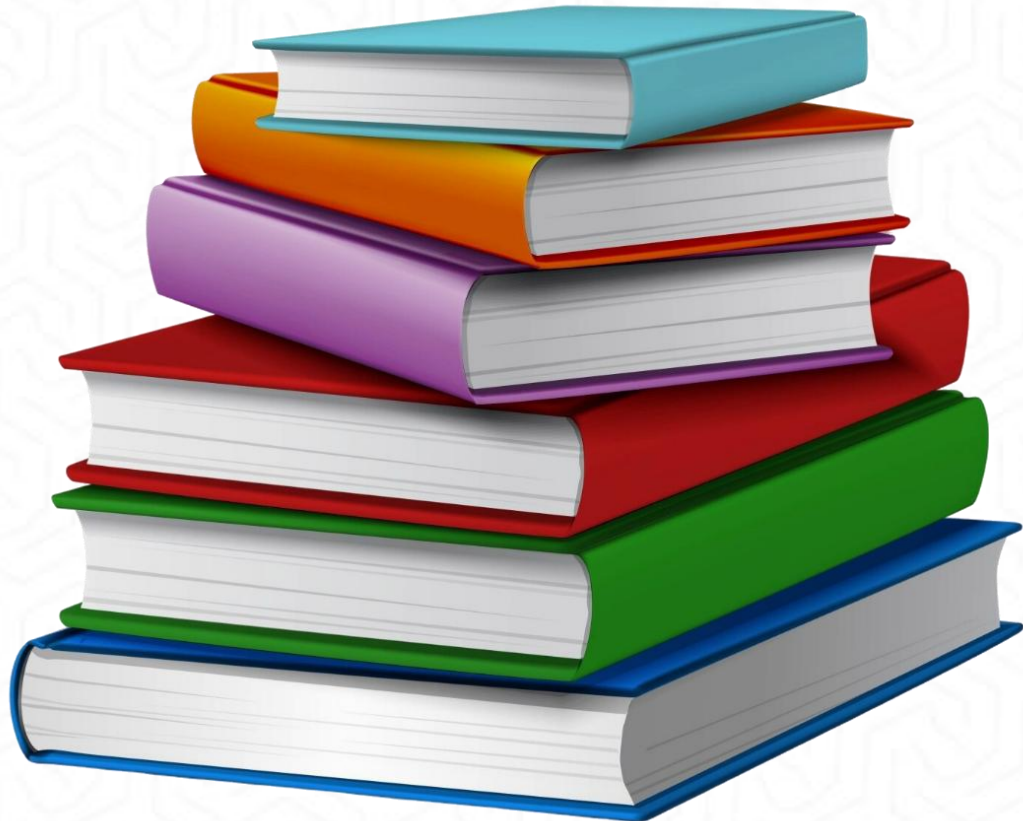
الدرجات:

92 : رياضيات

87 : علوم

89.50 : المعدل

2 : إجمالي عدد الطلاب



- Python for Everybody: Exploring Data in Python 3 Book by Charles Severance.



الكلية التطبيقية

الجامعة السعودية الإلكترونية

SAUDI ELECTRONIC UNIVERSITY

2011-1432

شكراً لكم





**الكلية التطبيقية**

**الجامعة السعودية الإلكترونية**

**SAUDI ELECTRONIC UNIVERSITY**

**2011-1432**

# دبلوم الذكاء الاصطناعي التوليدي

اسم المقرر: البرمجة باستخدام بايثون

رمز المقرر: PYT103

الموضوع : التاسع



# الموضوع التاسع: معالجة الأخطاء وإدخال/إخراج الملفات

# الأهداف

في نهاية هذا الدرس سيكون الطالب قادرًا على أن:

- ✓ يفهم آلية معالجة الأخطاء في Python
- ✓ يتقن استخدام بنية try/except
- ✓ يستخدم مع الملفات (فتح، قراءة، كتابة، إغلاق)
- ✓ يطبق أفضل الممارسات في إدارة الملفات
- ✓ يحل المشكلات الشائعة في إدخال/إخراج الملفات

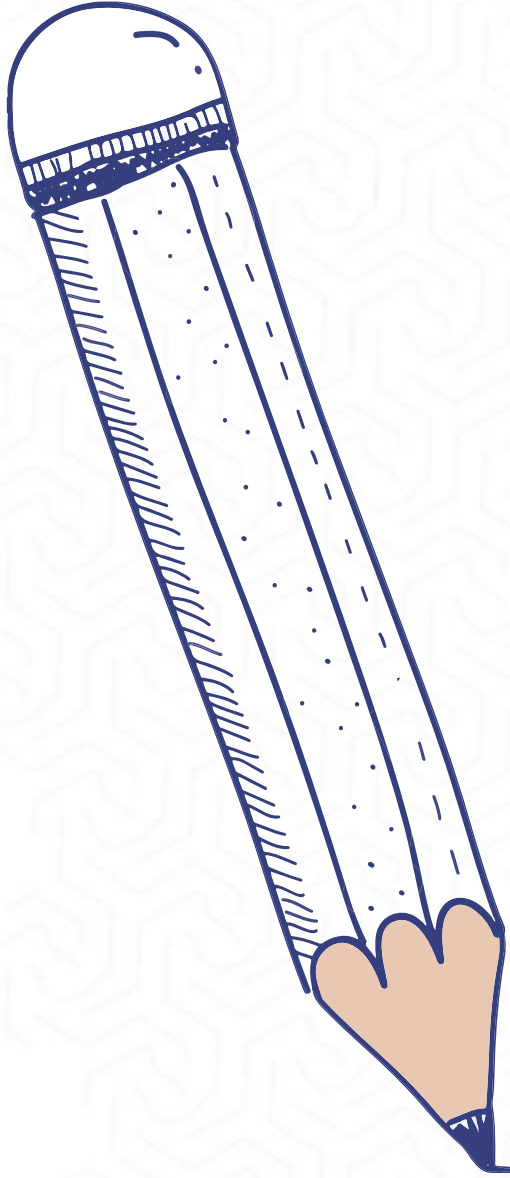




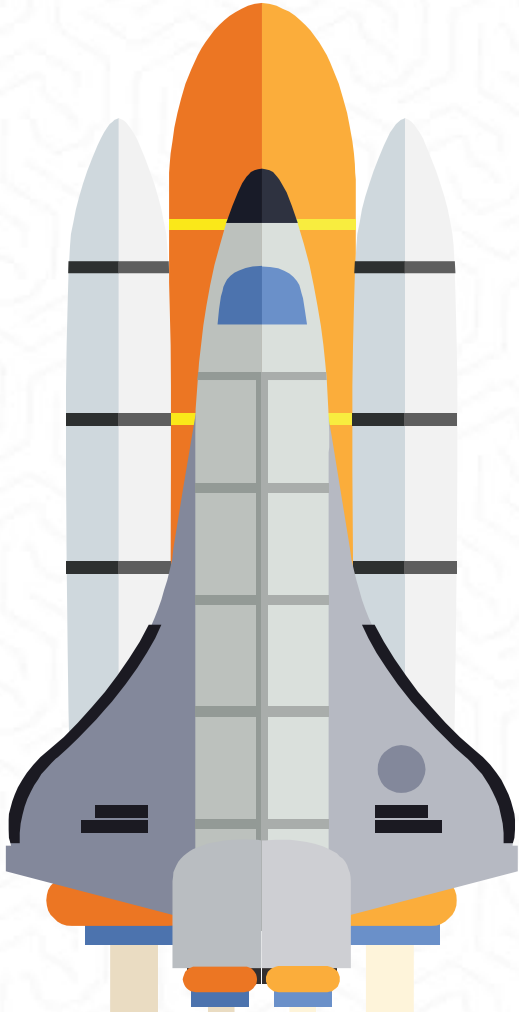
# وصف الدرس

في هذا الدرس سنتناول:

- كيفية بناء برامج قوية ومرنة قادرة على التعامل مع الأخطاء المحتملة بكفاءة.
- أساليب توقع الأخطاء ومعالجتها قبل أن تؤدي إلى توقف البرنامج.
- مهارات التعامل مع الملفات من حيث القراءة والكتابة بطرق آمنة وفعالة.
- استخدام تقنيات المعالجة الوقائية لضمان استقرار البرامج أثناء التنفيذ.
- تطبيقات عملية على إدارة الأخطاء والتعامل مع الملفات باستخدام لغة



## الموضوع التاسع | مقدمة عن معالجة الأخطاء



### • ما هي معالجة الأخطاء؟

- آلية للتعامل مع الأخطاء المحتملة أثناء تنفيذ البرنامج
- تمنع توقف البرنامج المفاجئ
- تسمح بتنفيذ إجراءات بديلة عند حدوث خطأ

### • أنواع الأخطاء:

- أخطاء قواعدية (Syntax Errors)
- أخطاء وقت التشغيل (Runtime Errors)
- أخطاء منطقية (Logic Errors)

### • لماذا نحتاج معالجة الأخطاء؟

- تحسين تجربة المستخدم
- منع فقدان البيانات
- تسهيل عملية التنقيح والصيانة

## الموضوع التاسع | بنية try/except الأساسية

### كيف تعمل بنية try/except:

try: يحتوي على الكود الذي قد يسبب خطأ

except: يحتوي على الكود الذي ينفذ عند حدوث الخطأ  
تسمح باستمرار تنفيذ البرنامج حتى لو حدث خطأ

```
try:
 # الكود الذي قد يسبب خطأ
 number = int(input("Enter a number: "))
 result = 10 / number
 print(f"Result: {result}")
except:
 # الكود الذي ينفذ عند حدوث أي خطأ
 print("تأكد من إدخال رقم صحيح وغير صفر! حدث خطأ")
```



## الموضوع التاسع | معالجة أنواع محددة من الأخطاء

```
try:
 # محاولة تحويل النص إلى رقم
 user_input = input("Enter a number: ")
 number = int(user_input)
 result = 100 / number
 print(f"100 divided by {number} = {result}")

except ValueError:
 # خطأ في تحويل النص إلى رقم
 print("يجب إدخال رقم صحيح فقط: خطأ")

except ZeroDivisionError:
 # خطأ القسمة على صفر
 print("لا يمكن القسمة على صفر: خطأ")

except Exception as e:
 # أي خطأ آخر
 print(f"حدث خطأ غير متوقع {e}")
```

التعامل مع أخطاء محددة:

يمكن تحديد نوع الخطأ المراد معالجته لتقديم

رسائل أكثر دقة



## الموضوع التاسع | استخدام finally و else

```
try:
 # محاولة فتح ملف
 file_name = input("Enter file name: ")
 file = open(file_name, 'r')
 content = file.read()

except FileNotFoundError:
 print("الملف غير موجود")

else:
 # ينفذ فقط إذا تم فتح الملف بنجاح
 print("تم قراءة الملف بنجاح")
 print(f"المحتوى: {content}")

finally:
 # ينفذ دائماً
 try:
 file.close()
 print("تم إغلاق الملف")
 except:
 print("لم يكن هناك ملف لإغلاقه")
```

- كتلة else:
- تنفذ فقط إذا لم يحدث أي خطأ في كتلة try
- كتلة finally:
- تنفذ دائماً، سواء حدث خطأ أم لا
- مفيدة لتنظيف الموارد

## الموضوع التاسع | مقدمة عن الملفات في Python

### • ما هي الملفات؟

- وسيلة لحفظ البيانات بشكل دائم
- تخزين في الذاكرة الثانوية (القرص الصلب)
- تبقى محفوظة حتى بعد إنهاء البرنامج

### • أنواع الملفات:

- ملفات نصية Text Files: تحتوي على نصوص قابلة للقراءة
- ملفات ثنائية Binary Files: تحتوي على بيانات مرمزة

### • العمليات الأساسية على الملفات:

- فتح الملف Open
- القراءة أو الكتابة Read/Write
- إغلاق الملف Close



## الموضوع التاسع | فتح الملفات - دالة open()

### كيف تعمل دالة open():

- تنشئ اتصالاً بين البرنامج والملف
- ترجع مؤشر ملف ( file handle ) للتعامل مع الملف
- تحتاج إلى اسم الملف ووضع الفتح

### أوضاع فتح الملفات الأساسية:

```
فتح ملف للقراءة
file_handle = open('data.txt', 'r')

فتح ملف للكتابة
output_file = open('output.txt', 'w')

فتح ملف للإضافة
log_file = open('log.txt', 'a')
```

- 'r': القراءة فقط (افتراضي)
- 'w': الكتابة فقط (يحذف المحتوى الموجود)
- 'a': الإضافة في نهاية الملف
- 'x': إنشاء ملف جديد (يفشل إذا كان موجوداً)

## الموضوع التاسع | قراءة الملفات - الطرق المختلفة

### طرق قراءة الملفات:

#### ١. قراءة الملف كاملاً:

# فتح الملف وقراءة كامل المحتوى

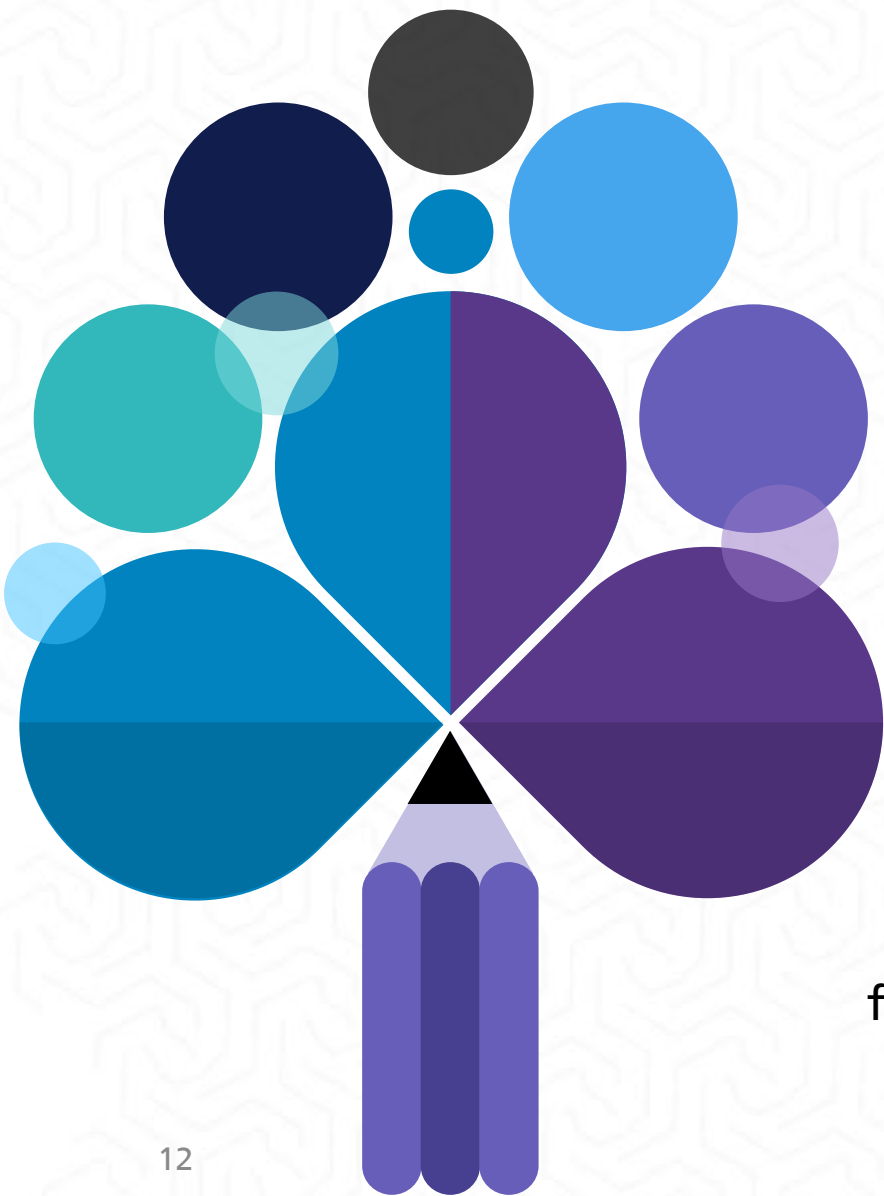
```
file = open('sample.txt', 'r')
content = file.read()
print(content)
file.close()
```

#### ٢. قراءة سطر واحد:

```
file = open('sample.txt', 'r')
first_line = file.readline()
print(f"السطر الأول: {first_line}")
file.close()
```

#### ٣. قراءة جميع الأسطر كقائمة:

```
file = open('sample.txt', 'r')
all_lines = file.readlines()
for i, line in enumerate(all_lines):
 print(f"السطر {i+1}: {line.strip()}")
file.close()
```





```
فتح الملف
file = open('students.txt', 'r')

التكرار عبر كل سطر في الملف
line_number = 1
for line in file:
 # إزالة رمز السطر الجديد من نهاية السطر
 clean_line = line.strip()
 print(f"الطالب {line_number}: {clean_line}")
 line_number += 1

إغلاق الملف
file.close()
```

استخدام حلقة for للمرور عبر

أسطر الملف:

هذه الطريقة أكثر كفاءة للملفات الكبيرة

لأنها تقرأ سطراً واحداً في كل مرة

```
file = open('grades.txt', 'r')
 total_grades = 0
 student_count = 0

 for line in file:
 grade = float(line.strip())
 total_grades += grade
 student_count += 1
print(f"درجة الطالب {student_count}: {grade}")

average = total_grades / student_count if
 student_count > 0 else 0
print(f"متوسط الدرجات: {average:.2f}")
file.close()
```

مثال متقدم –  
معالجة بيانات الطلاب:

### كيفية كتابة البيانات إلى الملفات:

#### ١. الكتابة الأساسية:

```
ينشئ ملف جديد أو يحذف المحتوى الموجود)فتح ملف للكتابة
output_file = open('results.txt', 'w')
```

```
كتابة نص إلى الملف
output_file.write("Python\nمرحباً بكم في برمجة")
output_file.write("\nهذا السطر الثاني")
```

```
إغلاق الملف
output_file.close()
```

```
قائمة من البيانات
student_names = ["محمود", "فاطمة علي", "أحمد محمد", "حسن"]
```

```
فتح الملف للكتابة
file = open('students_list.txt', 'w')
```

```
كتابة كل اسم في سطر منفصل
for name in student_names:
 file.write(f"{name}\n")
```

```
file.close()
```

#### ٢. كتابة قائمة من الأسطر:

```
إضافة محتوى جديد إلى ملف موجود
log_file = open('activity_log.txt', 'a')
```

```
إضافة إدخالات جديدة
from datetime import datetime
current_time = datetime.now().strftime("%Y-%m-%d
%H:%M:%S")
```

```
log_file.write(f"[{current_time}] المستخدم دخل إلى
النظام\n")
log_file.write(f"[{current_time}] أتم تنفيذ عملية جديدة\n")

log_file.close()
```

استخدام وضع 'a' للإضافة:

هذا الوضع يضيف المحتوى

الجديد إلى نهاية الملف دون

حذف المحتوى الموجود



```
إضافة درجات جديدة لملف موجود
grades_file = open('final_grades.txt', 'a')

قائمة الدرجات الجديدة
new_grades = [85, 92, 78, 96, 88]

for grade in new_grades:
 grades_file.write(f"{grade}\n")

print("تم إضافة الدرجات الجديدة بنجاح")
grades_file.close()
```

مثال عملي –

تسجيل درجات الطلاب:

## الموضوع التاسع | إدارة الملفات الآمنة مع with

```
(غير مُستحسنة) الطريقة التقليدية
file = open('data.txt', 'r')
content = file.read()
print(content)
file.close() # قد ننسى إغلاق الملف
```

```
with الطريقة الآمنة باستخدام
with open('data.txt', 'r') as file:
 content = file.read()
 print(content)
الملف يُغلق تلقائياً هنا
```

```
بطريقة آمنة CSV قراءة وكتابة ملف
with open('students.csv', 'r') as input_file:
 with open('processed_students.csv', 'w') as output_file:
 for line in input_file:
 # معالجة البيانات
 processed_line = line.upper().strip()
 output_file.write(f"{processed_line}\n")
```

### لماذا نستخدم with statement؟

- يضمن إغلاق الملف تلقائياً حتى لو حدث خطأ
- يجعل الكود أكثر وضوحاً وأماناً
- يتبع مبدأ "السياق" (Context Management)

مثال : الطريقة التقليدية مقابل with:

### مثال - معالجة ملف CSV:

## الموضوع التاسع | دمج معالجة الأخطاء مع إدارة الملفات

```
def read_student_grades(filename):
 """قراءة درجات الطلاب من ملف مع معالجة الأخطاء"""
 try:
 with open(filename, 'r') as file:
 grades = []
 for line_number, line in enumerate(file, 1):
 try:
 # تحويل كل سطر إلى رقم
 grade = float(line.strip())

 # التحقق من صحة الدرجة
 if 0 <= grade <= 100:
 grades.append(grade)
 else:
 print(f"تحذير {line_number}: درجة غير صحيحة في السطر {grade}")

 except ValueError:
 print(f"خطأ {line_number}: قيمة غير صحيحة في السطر {line.strip()}")

 return grades
 except FileNotFoundError:
 print(f"غير موجود {filename} الملف: خطأ")
 return []

 except PermissionError:
 print(f"{filename} لا توجد صلاحية لقراءة الملف: خطأ")
 return []

 except Exception as e:
 print(f"{e}: خطأ غير متوقع")
 return []

استخدام الدالة
grades = read_student_grades('grades.txt')
if grades:
 average = sum(grades) / len(grades)
 print(f"متوسط الدرجات: {average:.2f}")
```

الجمع بين try/except و with

للمحماية الكاملة:

## الموضوع التاسع | مثال تطبيقي- نظام إدارة الطلاب

```
def save_student_data(students, filename):
 """حفظ بيانات الطلاب في ملف"""
 try:
 with open(filename, 'w', encoding='utf-8') as file:
 file.write("رأس الجدول\nالدرجة,العمر,اسم الطالب")
 for student in students:
 line = f"{student['name']},{student['age']},{student['grade']}\n"
 file.write(line)
 print(f"تم حفظ بيانات {len(students)} طالب في {filename}")
 return True
 except Exception as e:
 print(f"خطأ في حفظ البيانات: {e}")
 return False

def load_student_data(filename):
 """تحميل بيانات الطلاب من ملف"""
 students = []
 try:
 with open(filename, 'r', encoding='utf-8') as file:
 # (رأس الجدول) تجاهل السطر الأول
 next(file)

 for line_number, line in enumerate(file, 2):
 try:
 # تقسيم السطر
 parts = line.strip().split(',')
 if len(parts) == 3:
 student = {
 'name': parts[0],
 'age': int(parts[1]),
 'grade': float(parts[2])
 }
 students.append(student)
 else:
 print(f"تحذير: صيغة خاطئة في السطر {line_number}")
```

```
except ValueError:
 print(f"خطأ في تحويل البيانات في السطر {line_number}")

except FileNotFoundError:
 print(f"غير موجود، سيتم إنشاء ملف جديد {filename} الملف")

except Exception as e:
 print(f"خطأ في تحميل البيانات: {e}")

return students

استخدام النظام
students = [
 {'name': 'أحمد محمد', 'age': 20, 'grade': 85.5},
 {'name': 'فاطمة أحمد', 'age': 19, 'grade': 92.0},
 {'name': 'محمود علي', 'age': 21, 'grade': 78.5}
]

حفظ البيانات
save_student_data(students, 'students.csv')

تحميل البيانات
loaded_students = load_student_data('students.csv')
print(f"طالب {len(loaded_students)} تم تحميل")
```



### أنواع المسارات:

- مسار مطلق: يبدأ من جذر النظام
- مسار نسبي: يبدأ من المجلد الحالي

```
import os

def safe_file_operations():
 """عمليات آمنة على الملفات مع التحقق من المسارات"""

 # التحقق من وجود مجلد
 data_folder = 'data'
 if not os.path.exists(data_folder):
 os.makedirs(data_folder)
 print(f"data_folder تم إنشاء مجلد")

 # بناء مسار الملف
 file_path = os.path.join(data_folder, 'students.txt')

 try:
 # التحقق من وجود الملف
 if os.path.exists(file_path):
 print(f"الملف موجود: {file_path}")

 # الحصول على معلومات الملف
 file_size = os.path.getsize(file_path)
 print(f"حجم الملف: {file_size} بايت")
 except:
 print(f"الملف غير موجود، سيتم إنشاؤه: {file_path}")
```

```
كتابة بيانات تجريبية
with open(file_path, 'w', encoding='utf-8') as file:
 file.write("أسماء الطلاب:\n")
 file.write("أحمد محمد 1.\n")
 file.write("فاطمة علي 2.\n")

print("تم إنشاء الملف بنجاح")

except Exception as e:
 print(f"خطأ في التعامل مع الملف: {e}")

تنفيذ العمليات
safe_file_operations()
```

# الموضوع التاسع | التعامل مع ملفات CSV

## ملفات CSV (Comma Separated Values):

- تنسيق شائع لتخزين البيانات الجدولية
- سهل القراءة والكتابة
- مدعوم في معظم البرامج

```
import csv
from datetime import datetime

def write_student_csv(filename, students):
 """كتابة بيانات الطلاب في ملف"""
 try:
 with open(filename, 'w', newline='', encoding='utf-8') as csvfile:
 # تحديد أعمدة الملف
 fieldnames = ['الاسم', 'العمر', 'الدرجة', 'المادة', 'التقييم_تاريخ']
 writer = csv.DictWriter(csvfile, fieldnames=fieldnames)

 # كتابة رأس الجدول
 writer.writeheader()

 # كتابة بيانات الطلاب
 for student in students:
 writer.writerow(student)

 print(f"طالب {filename} {len(students)} تم حفظ بيانات")

 except Exception as e:
 print(f"خطأ في كتابة ملف CSV: {e}")

def read_student_csv(filename):
 """قراءة بيانات الطلاب من ملف"""
 students = []
```

```
try:
 with open(filename, 'r', encoding='utf-8') as csvfile:
 reader = csv.DictReader(csvfile)

 for row_number, row in enumerate(reader, 1):
 try:
 student = {
 'الاسم': row['الاسم'],
 'العمر': int(row['العمر']),
 'الدرجة': float(row['الدرجة']),
 'المادة': row['المادة'],
 'التقييم_تاريخ': row['التقييم_تاريخ']
 }
 students.append(student)

 except ValueError as e:
 print(f"خطأ {row_number}: {e} في السطر")
 except KeyError as e:
 print(f"عمود مفقود في السطر {row_number}: {e}")

 print(f"طالب {filename} من {len(students)} تم تحميل")
 return students

 except FileNotFoundError:
 print(f"غير موجود {filename} الملف")
 return []

 except Exception as e:
 print(f"خطأ في قراءة ملف CSV: {e}")
 return []
```

```
بيانات تجريبية
students_data = [
 {'الاسم': 'أحمد محمد', 'العمر': 20, 'الدرجة': 85.5, 'المادة': 'Python', 'التقييم_تاريخ': '2024-01-15'},
 {'الاسم': 'فاطمة أحمد', 'العمر': 19, 'الدرجة': 92.0, 'المادة': 'Python', 'التقييم_تاريخ': '2024-01-15'},
 {'الاسم': 'محمود علي', 'العمر': 21, 'الدرجة': 78.5, 'المادة': 'Python', 'التقييم_تاريخ': '2024-01-15'}
]
```

```
كتابة البيانات
write_student_csv('students.csv', students_data)
```

```
قراءة البيانات
loaded_students = read_student_csv('students.csv')
```

```
عرض النتائج
for student in loaded_students:
 print(f"الطالب: {student['الاسم']}, الدرجة: {student['الدرجة']}")
```

ملخص ما تعلمناه:

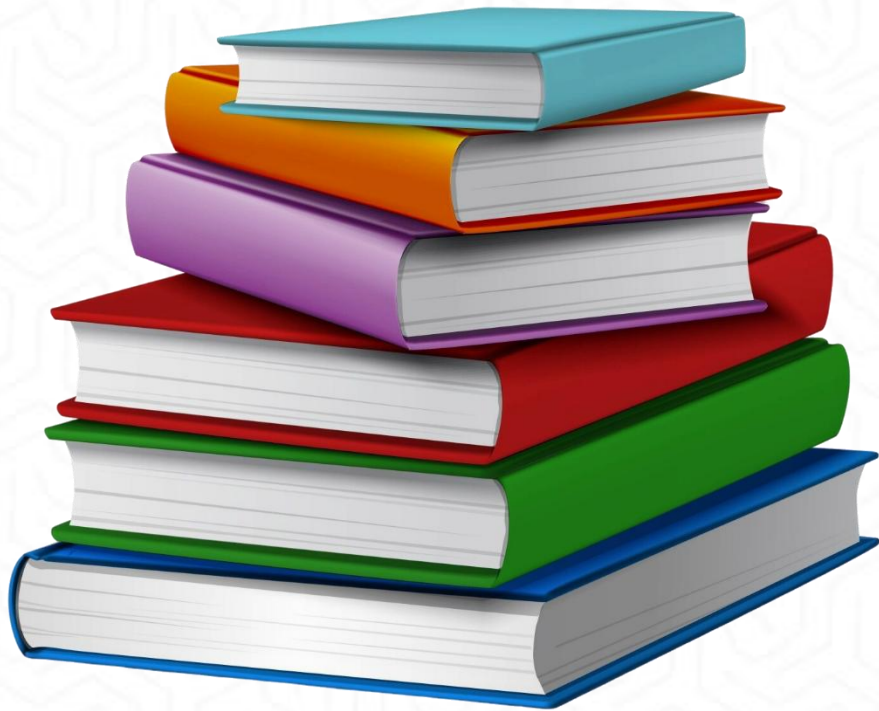
معالجة الأخطاء:

استخدام بنية try/except/finally  
التعامل مع أنواع محددة من الأخطاء  
أفضل الممارسات في معالجة الأخطاء

إدارة الملفات:

فتح وإغلاق الملفات بطريقة آمنة  
قراءة وكتابة البيانات  
التعامل مع ملفات CSV  
استخدام with statement





- Python for Everybody: Exploring Data in Python 3 Book by Charles Severance.





الكلية التطبيقية

الجامعة السعودية الإلكترونية

SAUDI ELECTRONIC UNIVERSITY

2011-1432

شكراً لكم



**الكلية التطبيقية**

**الجامعة السعودية الإلكترونية**

**SAUDI ELECTRONIC UNIVERSITY**

**2011-1432**

# دبلوم الذكاء الاصطناعي التوليدي

اسم المقرر: البرمجة باستخدام بايثون

رمز المقرر: PYT103

الموضوع: الحلقات (for, while)





# الموضوع السابع: الحلقات (for, while)

وزارة التعليم  
المملكة العربية السعودية



# الأهداف

في نهاية هذا الدرس سيكون الطالب قادرًا على أن:

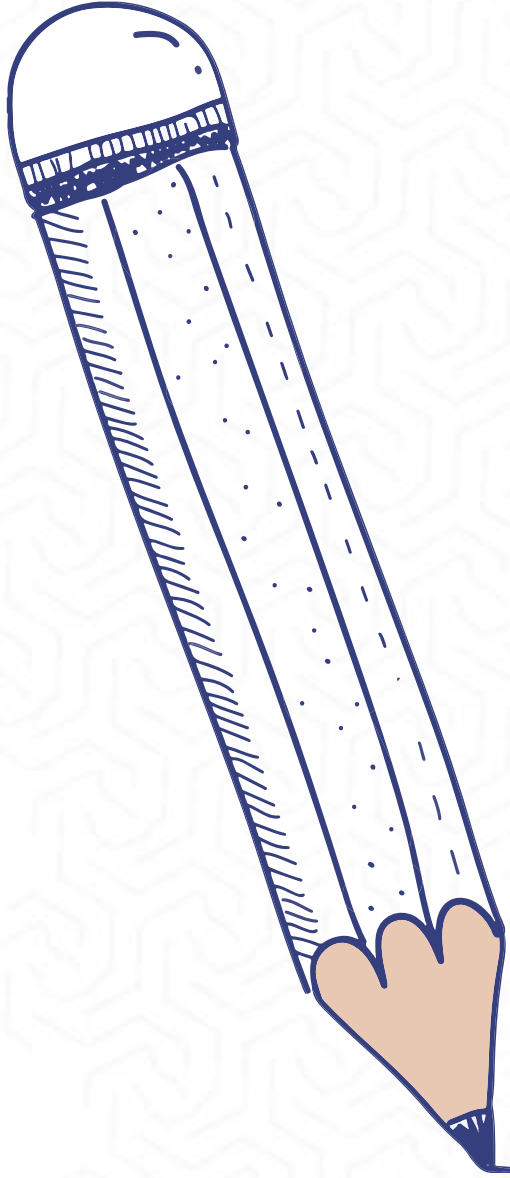
- ✓ يشرح مفهوم التكرار في البرمجة.
- ✓ يطبق حلقة while بطريقة صحيحة.
- ✓ يوظف حلقة for مع المجموعات المختلفة.
- ✓ يتجنب الوقوع في الحلقات اللانهائية.
- ✓ يستخدم أنماط البرمجة الشائعة مع الحلقات.
- ✓ يتحكم في تدفق الحلقات باستخدام break و continue.



# وصف الدرس

في هذا الدرس سنتناول:

- مفهوم التكرار في البرمجة باستخدام لغة Python.
- استخدام حلقتي `while` و `for` لتنفيذ العمليات المتكررة بكفاءة.
- أمثلة عملية متدرجة من البسيط إلى المعقد.
- تطبيقات عملية للحلقات في حل المشكلات البرمجية اليومية مثل معالجة البيانات والبحث والتحليل.



## الموضوع السابع | مقدمة عن التكرار

### لماذا نحتاج للحلقات؟

الحاسوب يبرع في تكرار المهام المتشابهة. 

تجنب كتابة نفس الكود مراراً وتكراراً. 

معالجة كميات كبيرة من البيانات بكفاءة. 

تنفيذ العمليات الحسابية المعقدة. 

### أنواع الحلقات في Python:

حلقة while حلقة شرطية 

حلقة for حلقة محددة 



## الموضوع السابع | مفهوم حلقة while

### حلقة: while

تستمر في التنفيذ طالما أن الشرط محقق True 

تفحص الشرط قبل كل تكرار 

مناسبة عندما لا نعرف عدد التكرارات مسبقاً 

### الصيغة العامة:

**while condition:**

# كود للتنفيذ

# تحديث متغير التحكم